



Snorkel AI

Project Terminus - New Expert Onboarding

March 2026

Project Overview

Project Terminus is Snorkel's effort to build a high-quality dataset in the style of [Terminal-Bench](#).

- Terminal-Bench is a benchmark for evaluating how well AI agents can accomplish complex tasks in a terminal environment
- Features multi-step tasks to be completed via a command line interface (CLI)
- Examples include:
 - Compiling and packaging a code repository
 - Downloading a dataset and training a classifier on it
 - Setting up a server

Your role is to create these tasks, along with an Oracle solution and associated tests that verify its correctness

- These tasks should be **challenging**, targeting a pass rate of <100% from SOTA models (e.g. GPT5.2, Claude Opus 4.6)
- Tasks will be created locally and submitted to the Snorkel Expert Platform where they will undergo CI validation, then peer review

Project Website

<https://snorkel-ai.github.io/Terminus-EC-Training-stateful/portal>

- Contains comprehensive project guidelines under “Docs” section
 - All documentation you need
 - Task Skeletons
- Task gallery where you can claim pre-generated task ideas to serve as basis of your task
 - Can use your own ideas as well

Task Difficulty Ratings

The following ratings will be applied to tasks to categorize their difficulty. Ratings are based on the accuracy of GPT-5.2 and Claude Opus 4.6 when attempting to complete the task.

Easy	Medium	Hard
>80% Accuracy	21-80% Accuracy	<= 20% Accuracy

NOTE: Since these ratings are based off model performance, you will not know how a task will be classified until after submitting.

If your task results in a “trivial” rating, you should iterate on the task to increase difficulty to be at least “easy”.

Task Components - Files

- **Task Instructions (instruction.md)**
 - Clear and self-contained description of the task to be accomplished for the AI agent
 - Natural tone - should represent the variety of prompting styles used by engineers
 - Variety - want different styles, tones, etc for each task. Try not to use same structure every time.
- **Task Description (task.toml)**
 - Configuration and metadata file
 - Uses TOML format with nested sections for task configuration, metadata, and environment settings.
- **environment/ Folder**
 - The environment definition must be placed in an environment/ folder. This prevents accidentally copying **task.toml**, **instruction.md**, or test files into the container.
 - *Required files based on environment type*
 - **For Docker Environment: environment/Dockerfile or environment/docker-compose.yaml**
 - **environment/Dockerfile**
 - Base image that fully sets up environment, including all required tools, resources, and dependencies
 - Should run without privileged mode
 - **Docker-compose.yaml**
 - Config file that defines orchestration for the task
 - Should reference base image from Dockerfile and include any necessary environment variables or mounted resources

Task Components - Files (cont.)

- **Oracle Solution (solution/solve.sh)**
 - Expert-authored, step-by-step solution contained within a shell script
 - Reliably and accurately completes the task
 - Task must pass Oracle solution to be accepted
- **Tests (tests/test.sh)**
 - Script to execute and validate end-to-end evaluation of task
 - The test script must be named tests/test.sh
 - This script installs test dependencies and verifies the agent completed the instruction.
 - The test script must produce a reward file in /logs/verifier/
- **Python tests (tests/test_outputs.py)**
 - Series of deterministic Python scripts containing unit tests
 - Checks task completion based on the final state of the environment
- **Rubric (done via Snorkel Platform submission interface)**
 - Do not include any rubric.txt in your .zip file, but is part of your overall submission package
- **Optional Components**
 - **Milestone descriptions ([milestones.md](#))**
 - Markdown file that succinctly describes each milestone of your task
 - Only include if your task has milestones
 - **Supporting data files**
 - Any additional inputs (e.g. data, config files, etc.) that are required for the task
 - **Custom tools**
 - Custom tools that can be used by the agent
 - Can be delivered in multiple ways, from source code files to companion containers
 - **Multiple containers**
 - Complex multi-service environments

Rubrics

Rubrics are structured, objective **binary checks over the agent's trace** that:

- Award points for **required, meaningful** steps that move the task forward.
- Penalize **incorrect, unsafe, or wasteful** steps.
- Provide a **graded** view of the trajectory quality beyond unit and functional tests.
- You generate rubrics during the submission UI via a checkbox
 - Once you submit for CI checks with checkbox checked, generated rubric will be present in submission
 - Make sure checkbox is NOT checked when sending to reviewer!
 - Overrides any current rubric you have
- ***You will then edit the rubric(s) to align with your current submission within the box in which it generated in***

Introducing Rubrics

Example rubric:

Agent includes basic sanity checks (e.g. validates magic bytes, breaks cleanly on short reads, avoids index errors) and reports errors clearly instead of silently failing, +1

Agent keeps all parsing logic data-driven (based on offsets and struct formats) and does not hardcode specific record values from the tests into the parser, +3

Agent uses destructive or dangerous shell commands unrelated to the task (e.g. `rm -rf /`, unbounded `find /` over the whole filesystem), -5

Milestones

Milestones divide a complex, multi-step engineering task into standalone, sequential subtasks.

- Design so that each stage is a prerequisite for the next.
 - The agent must explicitly signal completion of a milestone before the framework validates the environment and allows progression.

Milestone Components

1. **Instructions ([instruction.md](#)):** Describe each milestone of your task
2. **Verifiers (tests/):** Milestone tasks use unit test verifiers aligned to each stage.
 - a. *Deterministic Checks:* Each milestone verifier must trigger a pass/fail state.
 - b. *State Persistence:* The environment must remain stable between milestone validations.
3. **Rubrics:** Check that your rubric covers your milestones
4. **Metadata (task.toml):** You must specify the total count and provide a text description for every milestone
 - a. ***Number_of_milestones***
 - i. If no milestones, this should be set to '0'

Task Type Taxonomy - Categories

system-administration

build-and-dependency-manag
ement

data-processing

games

software-engineering

machine-learning

debugging

security

scientific-computing

Task Subtype Taxonomy - Subcategories

- **Task subtypes/subcategories**
 - Aligned to subtypes that target key challenge areas. Tasks can be aligned with multi-subtypes if they span multiple challenge areas.
 - **5 possible subtypes:**
 - *Long Context*
 - *Tool Specific*
 - *API Integration*
 - *DB Interaction*
 - *UI Building*
 - **subcategories** field in **task.toml**
 - If task aligns with one or multiple subcategories, list them here
 - If task does NOT align with any subcategories, leave this BLANK

General Aspects

- **Diversity in *codebase_size***
 - The size of the environment with regards to the number of files in the project.
 - **large:** Production or near production scale codebases that contain 200+ files of context. If forked from open source repos, name of the repo and commit should will be included in the task metadata.
 - **small:** Codebases that only contain a smaller set of 20+ files or scripts.
 - **minimal:** Tasks with little to no codebase context (0-20 files)included in the environment.
- **Diversity in coding *languages***
 - Want no more than 50% of this dataset to be Python
- **Tool Specific subcategory**
 - No more than 10% of tasks should be using FastAPI
 - Oversaturation of this tool otherwise
- **Diversity in *prompting style* in [*instructions.md*](#)**
 - Natural tone, mimic the diverse ways engineers interact with and prompt AI agents
- **Avoid *multi-container setups***
 - No more than 10% of tasks should be multi-container

Submission Quality Control

All submitted tasks are evaluated for quality and accuracy in three ways.

1. LLM-as-Judge (LLMaJ)

- a. Programmatic evaluators that run upon task submission
- b. Check for non-deterministic quality indicators (e.g. do the tests cover all intended behavior for the task?)
- c. Should iterate on submission until all of these pass

2. Deterministic Continuous Integration (CI)

- a. Programmatic checks that run upon task submission with "Send to Reviewer" **NOT** checked
- b. Check for deterministic adherence to structure and formatting (e.g. do all required files exist?)
- c. Should correct any issues that lead to CI failure

3. Manual Peer Review

- a. Once your task has passed both types of programmatic checks, it will be manually checked by an expert peer reviewer
- b. Peer reviewer will leave comments on the task indicating what needs to be added, removed, or corrected
- c. Once you submit with "Send to Reviewer" checked

LLMaJ - Part 1

- Behavior in Task Description
 - Checks whether all behaviors asserted by the tests are explicitly described in task.toml
 - Fails if the tests enforce implementation details that the task does not specify
- Behavior in Tests
 - Checks whether all behaviors described in the task.toml are actually tested (inverse of above)
 - Fails if one or more specified behaviors from the task is not fully tested
- Informative Test Docstrings
 - Checks for a clear and informative docstring describing each test
 - Fails if tests are missing docstrings or have poorly written ones
- Anti Cheating Measures
 - Checks that tests are executed after agent runs and there is no path to cheat by reading tests
 - Fails if there are ways for an agent to pass the tests by cheating instead of actually implementing functionality
- Structured Data Schema
 - If applicable, checks that any data files implied by the tests are specified in the task.toml or a separate schema file
 - Fails if any tests imply a data file that is not specified in the task

LLMaJ - Part 2

- Pinned Dependencies
 - Checks if required dependencies are pinned in the Dockerfile for reproducibility
 - Fails if any required dependencies are missing a pinned version
- Typos
 - Checks for typos in filenames, variables, and instructions
 - Fails if any typos are found
- Tests or Solution in Image
 - Checks to make sure that the Dockerfile does not copy tests or a solution into the image
 - Fails if Dockerfile does copy tests or solution into image
- Test Deps in Image
 - Checks to make sure that test dependencies are installed at test time (e.g., in tests/test.sh)
 - Fails if dependencies are installed into the image/during build
- Hardcoded Solution
 - Checks that the solution script writes a full source file implementing required behavior, not a hard-coded answer
 - Fails if the solution script merely echoes a final answer instead of implementing logic
- File Reference Mentioned
 - Checks that the required output file names given by the tests are specified in the task.toml
 - Fails if the task.toml does not mention the required output names that the tests check for

CI Evals - Part 1

- **Check Test File References**
 - **Operates on:** *test_outputs.py, task.toml*
 - **Checks:** all file references in *test_outputs.yaml* are mentioned in the *task.toml* - files that appear in both test and solution files but not in *task.toml* are flagged
 - **If fails:** either remove the files or add them to the task instructions
- **Validate Task Fields**
 - **Operates on:** *task.toml*
 - **Checks:** all required fields are present in the *task.toml*
 - **If fails:** add the missing required field
- **Check for Privileged Containers**
 - **Operates on:** *docker-compose.yaml*
 - **Checks:** for privileged containers in the *docker-compose.yaml*
 - **If fails:** remove any privileged container in the *docker-compose.yaml*

CI Evals - Part 2

- **Check PR Files**
 - **Operates on:** *tasks/*
 - **Checks:** for any files not in the *tasks/* directory
 - **If fails:** delete or move all files not in the *tasks/* directory
- **Check task file sizes**
 - **Operates on:** all files
 - **Checks:** that every file is under 1MB
 - **If fails:** any files over 1MB should be removed or condensed

CI Evals - Part 3

- **Ruff**
 - **Operates on:** all files
 - **Checks:** for Linting errors
 - **If fails:** fix any Linting errors
- **Check tests/test.sh Sanity**
 - **Operates on:** *tests/test.sh, task.toml*
 - **Checks:** if the tests/test.sh file uses uv init or has a task.toml file with global or system-wide keywords
 - **If fails:** update tests/test.sh to contain a uv init or uv venv, or add global or system-wide keywords to the task.toml

CI Evals - Part 4

- **Check Task Absolute Paths**
 - **Operates on:** *task.toml*
 - **Checks:** that task instructions use absolute paths rather than relative paths
 - **If fails:** update instructions to use absolute paths
- **Check Dockerfile References**
 - **Operates on:** *Dockerfile*
 - **Checks:** if any of `solution/solve.sh`, `tests/test.sh`, or `test_outputs.py` are in the Dockerfile
 - **If fails:** remove the forbidden file(s) from the Dockerfile

Task Design Requirements

- **Multi-Step**
 - Tasks requiring chaining multiple commands, handling intermediate states
 - Not solvable with a single command or episode of commands
- **Testable**
 - Must be fully specified so that agent can attempt to complete the task without ambiguity
 - Must be able to design unit tests to determine if final state of environment is correct
- **Unique**
 - Tasks should be unique and distinct from all existing tasks in the public Terminal-Bench benchmark
 - Should also be unique from other tasks submitted for this project
- **No Privileged Ops**
 - Tasks must not require root-level privileges or unsafe Docker settings like `--privileged`
- **Standalone**
 - Tasks must run to completion without additional user input after start
 - All parameters must be provided via files, flags, environment variables, or in the task.toml
- **Interacts with the Environment**
 - Looking for tasks that interact with the agent through the terminal
 - *For example: Avoid tasks based on data structures/algorithms.*
 - *OK for it to be a sub-component of the tasks, but shouldn't be the main component/focus*

Base requirements for a task to be accepted:

- All python unit tests all have to pass at least once across all 10 agent runs
- The Claude and ChatGPT agents cannot pass with 100% accuracy across tests
- Oracle solution needs to pass
- NOP shouldn't pass

Essential Resources

- Slack Channels:
 - [#terminus-2nd-edition-submission](#)
 - General channel for help, questions, discussion
 - [#terminus-2nd-edition-announcements](#)
 - Consolidated view of all important project announcements and updates
 - Only Snorkel can post.
- Project Website:
 - <https://snorkel-ai.github.io/Terminus-EC-Training-stateful/portal>
 - **Docs section**
 - Comprehensive project guidelines
 - 3 task skeletons available here
 - Also can be found in Slack channel [‘Resources’](#) section
 - **Task gallery**
 - Pre-generated task ideas you can claim and use to create your submissions
 - Can come up with your own original ideas too!

Completing a Task - Part 1

After you have followed the steps to create your task folder and downloaded correct task skeleton, do the following to complete your task:

1. Edit the created Dockerfile using a text editor to set up your task environment
 - a. Add any dependencies of the task, such as additional required packages
 - b. If you require a multi-container environment or other custom configuration, see [this page](#) for more information on how to customize your docker-compose.yaml
 - c. Docker Troubleshooting
 - i. Ensure you have a recent installation of Docker Desktop.
 - ii. On MacOS, enable the option in Advanced Settings: "Allow the default Docker socket to be used (requires password)."
 - iii. Try the following:
 1. `sudo dscl . create /Groups/docker`
 2. `sudo dseditgroup -o edit -a $USER -t user docker`
2. Enter your task container in interactive mode: `harbor tasks start-env --path <task-folder> --interactive`

Completing a Task - Part 2

Continued from previous slide:

3. While interacting with your task container, test your solution idea to make sure that it works as expected
 - a. Once solution is verified, record it and exit the container
4. Modify the solution file (solution/solve.sh) with the verified commands from the previous step
 - a. This file will be used by the OracleAgent to ensure the task is solvable
5. Update the tests/test_outputs.py file to verify task completion
 - a. Create pytest unit tests to ensure that the task was completed correctly
 - b. If tests require any file dependencies, place them in the tests/ directory
6. Test your task solution passes and meets all the requirements specified in the tests: `harbor run --agent oracle --path <task-folder>`
 - a. Note that you will need to install harbor from pypi in order to use these commands (`pip install harbor`)

Completing a Task - Part 3

Continued from previous slide:

7. Test your task solution with real agent
 - a. Receive API key and update using Snorkel CLI tool
 - b. Update environment variables
 - i. `export OPENAI_API_KEY=<Portkey API key>`
 - ii. `export OPENAI_BASE_URL=https://api.portkey.ai/v1`
 - c. Two models are available currently - GPT-5.2 and Claude Opus 4.6:
 - i. `uv run harbor run \`
`-a terminus-2 \`
`-m openai/"@openai-tbench/gpt-5-2" \`
`-p <task-folder>`
 - ii. `uv run harbor run \`
`-a terminus-2 \`
`-m openai/@anthropic-tbench/claude-opus-4-6 \`
`-p <task-folder>`
8. Run CI/LLMaJ locally on your task
 - a. `harbor run -a terminus-2 -m openai/@openai-tbench/gpt-5 -p <task-folder>`

Completing a Task - Part 4

Continued from previous slide:

9. Create a ZIP file of your task folder
10. Submit your task on the Snorkel Expert Platform in the **Terminus-2nd-Edition** project

High-Level Tasking Workflow - Submitting your Task

You will submit your assembled .zip file through the Terminus-2nd-Edition submission node on the Snorkel Expert Platform; LLMaJ checks, CI checks, rubric generation happen here.

Once you are granted access, you should:

1. Go to project website
2. Install `harbor` so that you can use the commands used for running the agents and programmatic checks locally (pip install harbor)
3. Come up with a task idea or select one from the [Task Gallery](#) (*note that you will still need to ensure these tasks pass and are hard enough, you may need to adjust them*)
4. Go to the [training site](#) and download the correct task file skeleton (3 options)
5. Rename the task folder to match your intended task name
6. Create your task instructions, an Oracle solution that passes, and Python tests
7. Iterate on your submission until all CI/Evals pass
8. Create a ZIP file for your task folder
 - a. *Select the underlying files to zip rather than zipping the high level folder*
9. Submit your ZIP file on the Platform

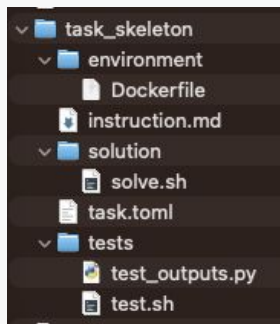
We expect submission tasks to take 2-5 hours based on task difficulty. Your first 1 or 2 will take longer, but much easier after initial learning curve

Platform Workflow - Pt. 1

- Download the ZIP file of the task skeleton
 - 3 options

```
my-task-folder/
├── instruction.md      # Concise, human-styled instructions
├── task.toml          # REQUIRED METADATA:
│                     # - task_type
│                     # - subcategories (if none, leave empty)
│                     # - difficulty
│                     # - codebase_size
│                     # - number_of_milestones (0 if no milestones)
│                     # - languages
│                     # - tags (3-6 keywords)
│                     # - runtime_limits (agent/verifier/build)
├── environment/       # Environment definition folder
│   ├── Dockerfile     # Standard single-container setup
│   ├── docker-compose.yml # Optional: For multi-container tasks
│   └── [build files]  # Any configs or scripts needed for the build phase
├── solution/          # Reference Oracle solution
│   └── solve.sh        # Step-by-step shell script that reliably solves task
├── tests/             # Test verification suite
│   ├── test.sh         # Main entry point for verification
│   └── test_outputs.py  # Deterministic state validation using Pytest
├── [optional] milestones.md # Only include if your task has milestones.
├── [optional files]    # Data files, assets, or sample code context
└── README.md          # Optional: Contributor notes or extra documentation
```

- Rename your task folder as desired, then implement your task locally (unchanged from GitHub repo flow). Your file structure will look something like this.



Platform Workflow - Pt. 2

- Go to your homepage on the Snorkel Expert Platform and click "Start" on the submission node for [Terminus-2nd-Edition](#)

My projects

2 projects available

terminus-2nd-edition

Newest First

Review

Start

Project: Terminus-2nd-Edition

Submission

Start

Project: Terminus-2nd-Edition

[Instructions](#)

Submission [Full instructions](#)

Exit task

☐ Rich doc

UID: 87139edd-f77a-45c2-aab5-0546c957809d



Task Notes



No notes available

Questions to answer

All form questions are required unless marked as optional.

Terminal bench 2.0 task submission



Upload terminal bench 2.0 submission here (zip file)

Zip file should have all files in the root, not under any folder.

Upload terminal bench 2.0 submission here (zip file) *



Upload file

Drag and drop or click to select

- This will load this Submission interface where you can drag and drop or browse to upload your task file

Platform Workflow - Pt. 3

- After uploading the file and clicking the "Check Feedback" button, the fast static checks will run on your file and return a summary of passes and failures
 - Note that this takes a minute or two to run
- If the checks all pass (green), you can continue to the next step
- If some of your checks fail (red), you **need to iterate on your task locally and then re-upload and run the checks again**

Upload terminal bench submission here (zip file)

Zip file should have all files in the root, not under any folder.

Upload terminal bench submission here (zip file) *

good_task.zip
11/25/2025, 12:20:09 PM

Fast static checks

Run quick static checks to verify submission structure and files. Submit runs slower checks and agent runs and may take a couple of minutes before returning results.

Clear feedback results

AutoEval Execution Summary

AutoEval execution succeeded. Build status: SUCCEEDED. Build ID: CodeExecutionEnvironment:ae1ef2a5-aa13-4ce4-9c12-3ef2a0a98640.

Show Build Logs

Build Logs

```
33 ✓ All provided paths are under a single task folder: tasks/tbench-task
34 ✓ Canary string present in all relevant files
35 ✓ Dockerfiles are clean of forbidden file references
36 ⚠️ [1:33m]Warning: /root/tasks/tbench-task/Dockerfile doesn't clean up apt cache
37 ✓ All 1 Dockerfiles passed the sanity check
38 ✓ All 1 run-tests.sh files passed the sanity check
39 ✓ All 1 tasks use proper absolute paths
40 ✓ All 1 tasks have properly documented file references
41 ✓ All files within provided task directories are within size limit (1048576 bytes).
```

Clear feedback results

AutoEval Execution Summary

AutoEval execution failed. Build status: FAILED. Build ID: CodeExecutionEnvironment:c734100e-85f9-4042-95aa-de9ceefaf730.

Show Build Logs

Build Logs

```
34 ✓ Canary string present in all relevant files
35 ✓ Dockerfiles are clean of forbidden file references
36 ✓ All 1 Dockerfiles passed the sanity check
37 ✗ 1/1 files failed the sanity check
38 ⚠️ [1:33m]/root/tasks/tbench-task/run-tests.sh
39
40
41 ✗ Use 'uv venv' or mark task.yaml with 'global'/'system-wide'
42 ✗ Issues found in /root/tasks/tbench-task (working dir: /app):
43 - Datastove path: /data/data.new - Should be: /app/data/data.new
```

Platform Workflow - Pt. 4

- Nothing will appear in these fields on the right at this stage - these will be used later
- Check generate rubric checkbox to have your rubric generated upon return of your submission from programmatic CI checks

Generate your Rubric(s) (optional)

Select this checkbox for rubric(s) to be
Once the evaluation completes, the ger
If you make significant changes to your

☐ Click here to generate rubric(s)

Checkbox (optional)

Check to send to reviewer if difficulty and quality checks are passing. Otherwise it will always result in revision

☐ Send to reviewer

Difficulty check results (optional)

Please disregard this field if it is blank, during initial submission. This field will be populated by the system when your code is run by the system.

Language: Python

Editor

1

Quality check results (optional)

Please disregard this field if it is blank, during initial submission. This field will be populated by the system when your code is run by the system.

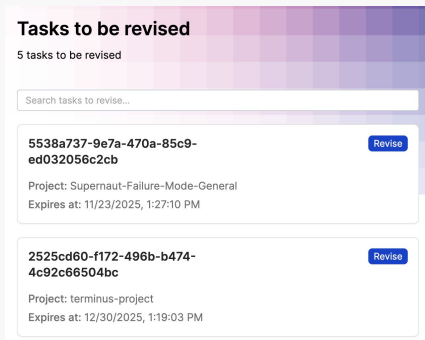
Language: Python

Editor

1

- Once all Fast Static Checks pass, click **Submit**.
- Do NOT check the 'Send to Reviewer' box at this point in time
- Sends your submission for programmatic CI checks

Platform Workflow - Pt. 5



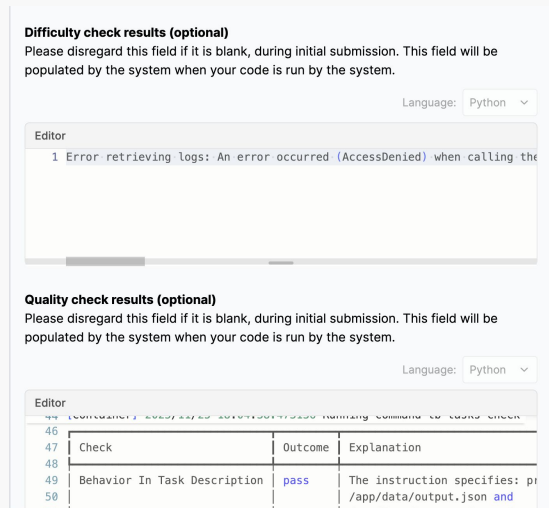
1. After submitting, after 10-15 minutes, the task will appear on the right side under "Tasks to be revised" - click Revise here

2. Now these two fields will be populated with the results of the agent runs and the LLM quality checks →

- If these don't pass, **you should iterate on your task and submit again**

3. Generated rubric will be present (if checkbox checked previously)

- Edit rubric for completeness and accuracy



Platform Workflow - Pt. 6

- **Once all checks pass**, and satisfied with rubric, send the task to human peer review by checking the *"Send to Reviewer"* box and then clicking **Submit**
- After a human reviews, they will either accept or send back for revisions - if they send back, it will again appear on the right side and you will need to iterate on your task and re-submit.
- Can dispute peer review feedback

Checkbox (optional)

Check to send to reviewer if difficulty and quality checks are passing. Otherwise it will always result in revision

☒ Send to reviewer

Submit

Logistics

- On platform project: *Terminus-2nd-Edition*
- Claim task ideas via website's task gallery section
- Use correct task skeletons, 3 of them:
 - "Regular" Task Skeleton
 - UI Task Skeleton
 - Milestone Task Skeleton
- Rate Schedule
 - \$250 per Accepted Submission
 - *Revisiting this soon!*
- Office Hours
 - *Get live help from Snorkel staff with your problems, any questions, whatever!*
 - *3 days a week currently (Mon, Wed, Fri)*